Week 5: Heaps, intervals, and graph traversal

Priorities and spreading failures

Maintain the top k measurements as values arrive, and merge overlapping maintenance intervals. On a sensor map, identify separate failure regions and the time needed for failures to spread.

Full ordering, partial ranking, and connectivity need different state. Track a heap boundary, the end of a sorted interval, and traversal visits.

Week 5 learning outcomes

Sessions 9 and 10: Heap boundaries, interval unions, visited sets, and BFS levels

- Check contracts and boundary conditions.
- Trace state and implement in Python.
- Explain correctness, time, auxiliary space, and output space.

Opening scenarios, traces, and checks are additional teaching examples.

Session 9 · Sorting, heaps, and intervals

Maintain top-k measurements and merge maintenance intervals. Separate questions requiring a full order from those requiring one priority boundary.

Prerequisites and Python semantics

sorted(…,key=lambda item:item[0]) sorts by first coordinate. heapq is a min-heap: heapify O(n), push/pop O(log n). Equal tuple priorities compare later fields; add a counter before nonorderable payloads.

Full sorting and partial ranking

- Full sort: O(n log n) time, all ranks available
- Size-k min-heap: retain only the top k
- New value ≤ root: keep the existing top k
- New value > root: replace the root and restore the heap

Import `heapq`. Heapify costs O(k), replacement O(log(k+1)); k=1 is supported.

215 · Problem and state

LeetCode 215: Kth Largest Element in an Array · Medium

Return the kth value in descending order, counting duplicates.

`nums=[8,3,8,2,5], k=2 → 8`

Heapify the first k values; replace the root only when a later value is larger.

215 · Implementation

```
def kth_largest(nums, k):
    heap = nums[:k]
    heapq.heapify(heap)
    for index in range(k, len(nums)):
        value = nums[index]
        if value > heap[0]:
            heapq.heapreplace(heap, value)
    return heap[0]
```

215 · Correctness and cost

The heap retains the largest k values seen. A new value at most its minimum cannot improve that set; a larger one replaces the minimum. The final root is kth largest.

n is length. O(n log(k+1)) time, O(k) space; heapify O(k). A full negated heap takes O(n+k log n) time and O(n) space.

56 · Problem and state

LeetCode 56: Merge Intervals · Medium

Return disjoint closed intervals covering the input union.

`intervals=[[5,7],[1,3],[3,6]] → [[1,7]]`

Sort by start; extend the last result end by max when intervals overlap.

56 · Implementation

```
def merge_intervals(intervals):
    result = []
    for start, end in sorted(intervals, key=lambda item: item[0]):
        if result and start <= result[-1][1]:
            result[-1][1] = max(result[-1][1], end)
        else:
            result.append([start, end])
    return result
```

56 · Correctness and cost

result equals the union of the processed prefix. Increasing starts mean a new interval can overlap only the last result; a gap makes a new disjoint component.

n is interval count. O(n log n) sorting plus O(n) scan; O(n) sorting workspace and output. Input order is preserved.

State trace

incoming value(k=2)	retained multiset	root
8,3	{3,8}	3
8	{8,8}	8
2	{8,8}	8
5	{8,8}	8

Each row retains the largest two values seen, including duplicates.

Failure inputs and concept checks

In a min-heap, each parent is no greater than its children, and the root holds a global minimum. Deduplicating changes the example’s answer.

Does insertion into a sorted list also cost log n?

The instructor answer is in the separate guide. Compare the reasoning line by line with the companion implementation.

Session 10 · Graph representations and simultaneous spread

Count connected sensor regions and propagation time from multiple failures. Use DFS or BFS to find connected regions, and use BFS levels to compute earliest arrival when all edges have equal propagation time.

Prerequisites and Python semantics

Represent a grid by coordinates, an adjacency list by vertex-to-neighbors, or an object graph by node.neighbors. Coordinate tuples are hashable; copy grid rows independently.

200 · Problem and state

LeetCode 200: Number of Islands · Medium

Count land components connected horizontally or vertically in a character grid.

`grid=[["1","0","1"],["1","0","0"]] → 2`

Start a stack traversal at each unseen land cell and mark its whole component.

200 · Implementation

```
def islands(grid):
    rows, cols = len(grid), len(grid[0])
    seen, count = set(), 0
    for row in range(rows):
        for col in range(cols):
            if grid[row][col] != "1" or (row, col) in seen:
                continue
            count += 1
            stack = [(row, col)]
            seen.add((row, col))
            while stack:
                r, c = stack.pop()
                for nr, nc in ((r-1,c),(r+1,c),(r,c-1),(r,c+1)):
                    if (0 <= nr < rows and 0 <= nc < cols
                            and grid[nr][nc] == "1" and (nr,nc) not in seen):
                        seen.add((nr,nc))
                        stack.append((nr,nc))
    return count
```

200 · Correctness and cost

A traversal reaches exactly its starting component. Marked components cannot restart; the number of starts equals the component count.

R,C are dimensions. O(RC) time and space; general adjacency lists take O(V+E) time.

994 · Problem and state

LeetCode 994: Rotting Oranges · Medium

Return the minimum minutes for all fresh cells to change under simultaneous spread, or -1.

`grid=[[2,1,1],[0,1,0]] → 2`

Enqueue all sources together, process levels, and mark a discovered cell as 2 immediately.

994 · Implementation

```
def rot_minutes(grid):
    rows, cols = len(grid), len(grid[0])
    board = [row[:] for row in grid]
    queue, fresh, minutes = deque(), 0, 0
    for r in range(rows):
        for c in range(cols):
            if board[r][c] == 2:
                queue.append((r,c))
            fresh += board[r][c] == 1
    while queue and fresh:
        for _ in range(len(queue)):
            r, c = queue.popleft()
            for nr, nc in ((r-1,c),(r+1,c),(r,c-1),(r,c+1)):
                if 0 <= nr < rows and 0 <= nc < cols and board[nr][nc] == 1:
                    board[nr][nc] = 2
                    fresh -= 1
                    queue.append((nr,nc))
        minutes += 1
    return minutes if fresh == 0 else -1
```

994 · Correctness and cost

BFS levels are minimum edge distances from any source. Equal edge durations make first discovery earliest; remaining fresh cells after exhaustion are unreachable.

R,C are dimensions. O(RC) time and O(RC) board-copy/queue space.

State trace

minute	current frontier	newly marked
0	(0,0)	(0,1)
1	(0,1)	(0,2),(1,1)
2	(0,2),(1,1)	none

The newly marked cells belong to time t+1, not time t.

Failure inputs and concept checks

Use == for values; is tests identity. Diagonal land in a 2x2 grid forms two components.

Does BFS guarantee minimum cost with weighted edges?

The instructor answer is in the separate guide. Compare the reasoning line by line with the companion implementation.

Time boundaries in simultaneous spread

Time	Newly rotten at that time	Fresh cells left
0	(0,0), initial source	3
1	(0,1)	2
2	(0,2), (1,1)	0

Grid: `[[2,1,1],[0,1,0]]`. One iteration over a fixed queue length advances one minute. Mark cells when enqueuing them.

Checkpoint questions

1. Find the answers for k=2 and k=5 in `[8,3,8,2,5]`.
2. Merge `[[1,8],[2,3]]` and, separately, `[[1,3],[3,6]]`.
3. Count components in `[["1","0"],["0","1"]]`.
4. Find spread times for `[[2,0,1]]` and `[[0,2]]`.

Independent practice and connections

Student exercises

Hints and solutions

Instructor guide

Download Week 5 code

Submit code, state definition, correctness argument, complexity, and boundary inputs. Week 8 includes a mixed assessment without pattern labels.

Execution and sources

Run in the `code` folder of the weekly code archive.

```
python demo_09.py
python demo_10.py
python verify.py
python verify.py --student
```

Use the last command after implementing the student file.

Erik Demaine and Srini Devadas, MIT 6.006: Introduction to Algorithms (2011).